



فريق موظفي الذكاء الاصطناعي

مرجع كامل لفريق عمل يبنّي، يراجع، ويؤمّن أي مشروع برمجي — من فكرة بسطر واحد إلى إصدار جاهز. كل موظف خبير مستقل، عامّ يتأقلم مع أي مشروع.

3

مستويات تشغيل

7

أقسام

32

موظف خبير

كيف يعمل الفريق — ثلاثة مستويات

تنادي موظفاً واحداً لمهمة دقيقة، أو قسماً كاملاً بأمر واحد، أو تشغل الشركة كلها من فكرة إلى إصدار.

/feature-factory

الشركة كلها: نقد → تخطيط → تنفيذ → فحوص → إصلاح → إطلاق

/team <القسم>

قسم كامل بأمر واحد — مثل security-team/

/اسم-الموظف <

موظف واحد لمهمة محددة — مثل bug-hunter

● قسم التخطيط /plan-team

من فكرة خام إلى مخطط تقني جاهز — قبل كتابة أي كود.

/codebase-researcher [مستكشف الكود]

يرسم خريطة الجزء المعني من الكود قبل أي تعديل: الملفات، الأنماط، ميزات مشابهة، والمخاطر.

للقراءة فقط: أول خطوة في أي ميزة

/idea-critic [ناقد الأفكار]

يحكم على الفكرة قبل أي بناء: هل موجودة بالسوق؟ من المنافسون؟ هل الطلب حقيقي؟ ثم حكم واضح.

الحكم: GO / RESHAPE / KILL

/spec-writer [كاتب المخطط التقني]

يحول القصة إلى مواصفة تقنية قصيرة: نموذج البيانات، التدفق، API، الواجهة، المخاطر، والملفات المتأثرة.

المخرج: مخطط تقني للبيئات

/story-writer [كاتب القصة]

يحول الفكرة إلى قصة مستخدم واضحة بمعايير قبول، حالات حدية، وما هو خارج النطاق.

المخرج: قصة + معايير قبول

● قسم التنفيذ /build-team

يبنى الميزة من المخطط — طبقة البيانات والمنطق، ثم الواجهة.

/frontend-builder [بناء الواجهة]

ينفذ نصف الواجهة: المكونات، الستايل، حالات التحميل/الخطأ/الفراغ، والترجمات إن كان المشروع متعدد اللغات.

يطابق أنماط الواجهة الموجودة

/backend-builder [بناء الداتا والمنطق]

ينفذ النصف غير المرئي من الميزة: التخزين، الخدمات، APIs، المزامنة، والمهام الخلفية — بأسلوب المشروع.

يتحقق باختبارات المشروع أو بتشغيله

/admin-panel-builder [بناء لوحة التحكم]

يبنى لوحة تحكم / باك-أوفيس احترافية مربوطة بكل كيان: نظرة عامة + قوائم/بحث/فلتر/تفاصيل/CRUD خلف بوابة أدمن.

أمان: المفتاح العام فقط — لا مفتاح مميّز في العميل أبداً

/ui-designer [مصمم الواجهات]

يقترح اتجاهاً بصرياً ويبنيه بإتقان: تسلسل، مسافات، لون، طباعة، حركة. للشاشات الجديدة والتصميم الجاد.

بدلاً من بناء الواجهة عند الحاجة لجمال حقيقي

/data-migration- guardian [حارس بيانات المستخدمين]

يحمي بيانات المستخدمين المخزنة عند أي تغيير في السكيمات؛ يكتب
ترحيلاً آمناً لا يكرر ولا يفقد صفاً.

له **فيتو** على فقدان البيانات (أعلى أولوية)

يمسك الأخطاء قبل المستخدم — صحة، تصميم، وصولية، أداء، هيكلية.

/implementation-validator [مدقق التنفيذ]

يقارن الكود الفعلي على القرص بالقصة والمخطط، ويطلع الفجوات مرتبة بالخطورة.

يُشغّل بعد البناء، قبل الإطلاق

/bug-hunter [صيّاد الأخطاء]

يصطاد أخطاء المنطق التي يفوّتها غيره: حالات حدّية، حالة قديمة، async مكسور، off-by-one.

للقراءة فقط

/accessibility-specialist [خبير الوصولية]

التباين اللوني، حجم أهداف اللمس، الوصول بلوحة المفاتيح، دلالات قارئ الشاشة، وإدارة التركيز.

يجعل الواجهة صالحة للجميع

/design-system-reviewer [مراجع التصميم]

يتأكد أن الواجهة الجديدة متّسقة مع نظام التصميم: tokens، المكونات، الحالات، responsive، RTL، dark.

عند أي تغيير واجهة

/refactoring-specialist [خبير إعادة الهيكلة]

يحسّن بنية الكود دون تغيير سلوكه: تقسيم ملفات كبيرة، حذف تكرار، توضيح أسماء — يتحقّق بعد كل خطوة.

السلوك يبقى مطابقاً

/performance-optimizer [محسّن الأداء]

أداء التطبيق: وقت الإقلاع، تكلفة الرسم، حجم الحزمة والأصول، والعمل المتزامن الثقيل. يقيس أولاً.

الطبقة: التطبيق (لا القاعدة)

فحص أمني شامل بعقلية OWASP — والاختير يحاول يكسر فعلاً.

/secrets-sentinel [حارس الأسرار]

يصطاد المفاتيح والأسرار المسرّبة في كود العميل وفي تاريخ git؛ الأسرار تبقى بالخادم، والعميل بمفتاح عام آمن فقط.

يفحص الحزمة + git history

/security-reviewer [مراجع الأمان]

تدقيق كودي بعقلية OWASP: الحقن، XSS، الصلاحيات، الأسرار، عزل البيانات، وتسريب الأخطاء.

للقراءة فقط

/pentester [مختبر الاختراق]

اختبار هجومي مصرّح لتطبيقك أنت فقط (دفاعي): جرّب الحقن، العبث بالحالة، وIDOR — ويعطي إعادة إنتاج للثغرة.

نطاق: تطبيقك وحده · لا هدف آخر

/access-control-auditor [مدقق الصلاحيات]

طبقة API/التطبيق: هل يقدر المستخدم A أن يصل ببيانات المستخدم B؟ يدقّق التصريح وتدفقات الدخول والاسترجاع.

السؤال: can user A read user B?

يثبت أن الميزة تعمل فعلاً في التطبيق الحيّ — بكل اللغات.

/test-verifier [مُتحَقِّق القبول]

يؤكد أن كل معيار قبول متحقق، موثّقاً بصيغة Given/When/Then بأدلة من التطبيق الفعلي.

يكتب اختبارات إن كان للمشروع إطار

/browser-verifier [فاحص المتصفح]

يشغّل التطبيق فعلياً ويثبت أن التغيير يعمل من طرف لطرف — المسار السعيد، الخطأ، والفراغ، وكل لغة مدعومة.

لا قراءة كود — سلوك حقيقي

/i18n-checker [فاحص الترجمة]

يتأكد أن كل نص للمستخدم يمرّ عبر طبقة الترجمة وله مدخل في كل اللغات — لا نص مكتوب في الكود.

يعمل فقط لو المشروع متعدد اللغات

يوصل البناء الجديد للمستخدمين فعلاً — نسخة، توثيق، ملاحظات إصدار.

/docs-updater [محدّث التوثيق]

يحدّث توثيق المشروع والذاكرة بعد كل تغيير: واجهات جديدة، gotchas، أرقام نسخ، وملفات جديدة.

يكتب التوثيق فقط — لا كود المنتج

/release-cache-manager [مدير الإصدار والكاش]

ينقذ عملية الإصدار وكسر الكاش في كل الأماكن المطلوبة، حتى يصل البناء الجديد للمستخدمين بدل خدمة كود قديم.

يمنع الكاش القديم من إخفاء التحديث

/pr-writer [كاتب الإصدار والPR]

يقرأ diff الحقيقي ويكتب رسالة الكوميت، وصف الPR، ملاحظات الإصدار، وقائمة النشر.

يقرأ git فقط — لا يغيّر كوداً

🔒 طبقة أمان بنىوية للأقسام كلها

عمليات قواعد البيانات المدمّرة (حذف جداول/بيانات) محكومة بـ **hook خارج النموذج** يفحص كل أمر قبل تشغيله: يجب أي عملية مدمّرة على قاعدة إنتاجية بلا تجاوز، ويسمح فقط للقواعد المؤقتة للتجربة، والتأكيد البشري إلزامي قبل أي عملية لا رجعة فيها — لا يُمنح نيابةً عن المستخدم أبداً.

يصمم قاعدة بيانات خادمة، يثبت تطبيعتها، يؤمنها ويفهرسها، ثم يطبقها تحت بروتوكول أمان صارم.

/normalization-auditor [مدقق التطبيع]

يستخرج الاعتماديات الوظيفية ويثبت الشكل الطبيعي الفعلي لكل جدول (حتى BCNF)، ويصحح ما دونه.

الهدف: BCNF · الأرضية: 3NF

/db-architect [مهندس السكيمات]

يحوّل المتطلبات إلى كيانات وعلاقات ومفاتيح وقيود، ويختار المنصة الخادمة المناسبة. يُخرج ERD ومسودة DDL.

قاعدة: القاعدة على خادم دائماً

/db-migration-engineer [مهندس الترحيلات]

يكتب ويطبق الترحيلات — حتى على الإنتاج — تحت بروتوكول: نسخة موثقة ← تجربة ← تغليف ← وقفة قبل أي حذف.

صفر توقف عبر expand→contract

/db-index-optimizer [مهندس الفهارس والأداء]

يسرّع الاستعلامات بأدنى مجموعة فهارس (بقياس EXPLAIN)، ويدير تجميع الاتصالات ومراقبة صحة القاعدة.

يقيس قبل وبعد — لا تخمين

/db-seed-engineer [مهندس البذور]

بذور مرجعية idempotent + بيانات تجريبية محروسة لا تصل الإنتاج + نسخ إنتاج مُجهّلة الهوية للتطوير المحلي.

لا تكرر عند إعادة التشغيل

/db-security-auditor [مدقق أمان القاعدة]

العزل داخل القاعدة نفسها: RLS، صلاحيات least-privilege، دوال أمنة، PII، واكتمال حذف حساب المستخدم.

يثبت العزل بمحاولة قراءة متقاطعة

/db-backup-engineer [مهندس النسخ الاحتياطي]

يأخذ نسخاً ويتحقق منها بالاسترجاع الفعلي؛ يعرّف RPO/RTO والجدولة، ويجري تدريبات استرجاع.

مبدؤه: نسخة لم تُسترجع = غير موجودة

أوامر سريعة

الأمر	ماذا يفعل
/feature-factory «فكرة»	الشركة كاملة: نقد → تخطيط → تنفيذ → فحص → إصلاح تلقائي → إطلاق
/plan-team	قسم التخطيط: ناقد → مستكشف → قصة → مخطط تقني
/build-team	قسم التنفيذ: حارس البيانات → الباك → الواجهة/التصميم
/review-team	قسم المراجعة: أخطاء + تنفيذ + تصميم + وصولية (بالتوازي)
/security-team	قسم الأمان: مراجع + أسرار + صلاحيات + مختبر اختراق
/verify-team	قسم التحقق: متصفح + قبول + ترجمة
/release-team	قسم الإطلاق: نسخة + توثيق + PR
/database-team	قسم القاعدة: تصميم → تطبيع → فهرسة/أمان/بذور → نسخة → ترحيل

أي موظف بمفرده لمهمة دقيقة	<اسم-موظف> /
----------------------------	--------------

32 موظف 7٠ أقسام ٠ فريق عام يتأقلم مع أي مشروع — مرجع للطباعة والمشاركة.